GM - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continous color palettes
```

```
# load needed functions
# source(paste0(RootDir,"/src/function_library/sp_functions.R"))
# do this in the calling R script
```

```
# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat," - ",CurTask," - ",RunLabel," - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE
```

```
CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres
```

```
# done in calling script
# print(paste0("Using root dir: ",RootDir))
# RunDir <- paste0(paste0(RootDir,"/output/",RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir,showWarnings = TRUE)
#   print(paste0("created run directory: ",RunDir))
# }
```

```
# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir,showWarnings = TRUE)
#   print(paste0("created participant directory: ",PatientDir))
# }
# TablesDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir,showWarnings = TRUE)
#   print(paste0("created tables directory: ",TablesDir))
# }
# FigDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir,showWarnings = TRUE)
#   print(paste0("created figures directory: ",FigDir))
# }
# ReportsDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir,showWarnings = TRUE)
#   print(paste0("created reports directory: ",ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())
```

```
ModelDatFilename<-paste0(RootDir,"/data/",CurPat,"/",CurPat,"_",CurTask,"_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ",ModelDatFilename))
  print(sprintf("\troot dir: ",RootDir))
}
PosDat<-read.csv(ModelDatFilename)
```

```
# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values",OrigDataLen))
```

```
## [1] "Original data has 4435 values"
```

```
if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values",DataLen))
  print(sprintf("%d values eliminated.",OrigDataLen - DataLen))
}
```

```
## [1] "Data without fractional preserved values has 4398 values"
## [1] "37 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite

# get counts of syllable components in different serial positions
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syl

syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))

kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	557	35	129	NA	NA	721
2	67	NA	444	95	111	717
3	312	NA	172	214	16	714
4	306	NA	237	69	39	651
5	232	NA	216	73	39	560
6	210	1	137	73	22	443
7	179	NA	105	28	19	331
8	92	NA	55	26	4	177
9	76	NA	2	NA	6	84

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7725381	0.0485437	0.1789182	NA	NA	721
2	0.0934449	NA	0.6192469	0.1324965	0.1548117	717
3	0.4369748	NA	0.2408964	0.2997199	0.0224090	714
4	0.4700461	NA	0.3640553	0.1059908	0.0599078	651
5	0.4142857	NA	0.3857143	0.1303571	0.0696429	560
6	0.4740406	0.0022573	0.3092551	0.1647856	0.0496614	443
7	0.5407855	NA	0.3172205	0.0845921	0.0574018	331
8	0.5197740	NA	0.3107345	0.1468927	0.0225989	177
9	0.9047619	NA	0.0238095	NA	0.0714286	84

```
# get percentages only from summary table
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
syll_comp_perc <- syll_comp_perc %>% rename("Coda" ="1","Satellite"="S")

# pivot to long format for plotting
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
            names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda","Sa
```

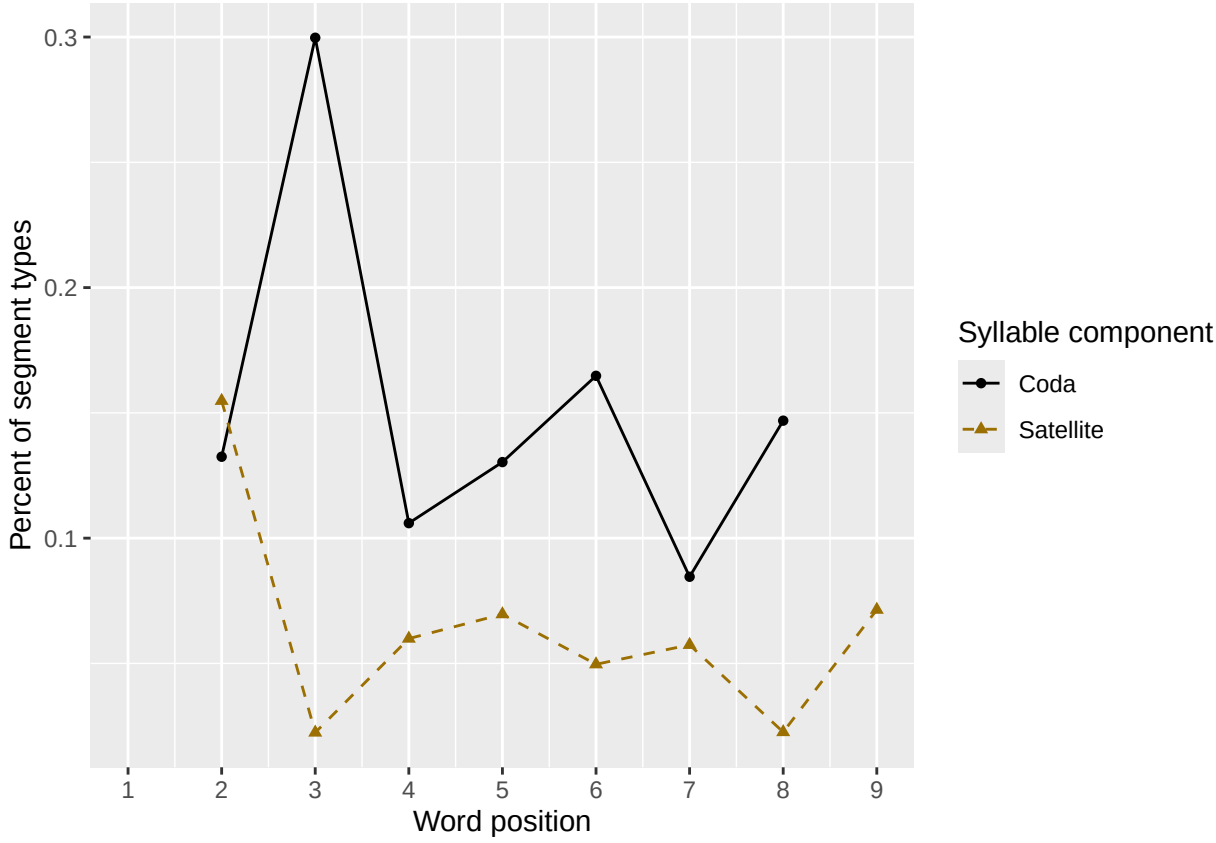
```
# plot satellite and coda occurances across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                         aes(x=pos,y=percent,group=syll_component,
                             color=syll_component,
                             linetype = syll_component,
                             shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_pos_of_syll_complexities_in_stimuli.png"),plot=sy
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`)
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```




```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```
min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,"_preserved_percentages_by_len_
pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.983 1     0.95  NA     NA     NA     NA     NA     NA
## 2       5 1     0.968 1     0.958 NA     NA     NA     NA     NA
## 3       6 1     0.975 1     0.967  0.917 NA     NA     NA     NA
## 4       7 1     0.991 1     0.973  0.974  0.965 NA     NA     NA
## 5       8 0.974 0.987 0.974 0.947  0.901  0.941  0.897 NA     NA
## 6       9 0.989 0.978 0.956 0.967  0.967  0.957  0.968  0.968 NA
## 7      10 0.988 0.988 0.951 0.963  0.939  0.952  0.928  0.940  0.917
```

```
# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved),N=n()) %>% dply
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                 "_N_by_len_position.csv"))
pos_len_N_table
```

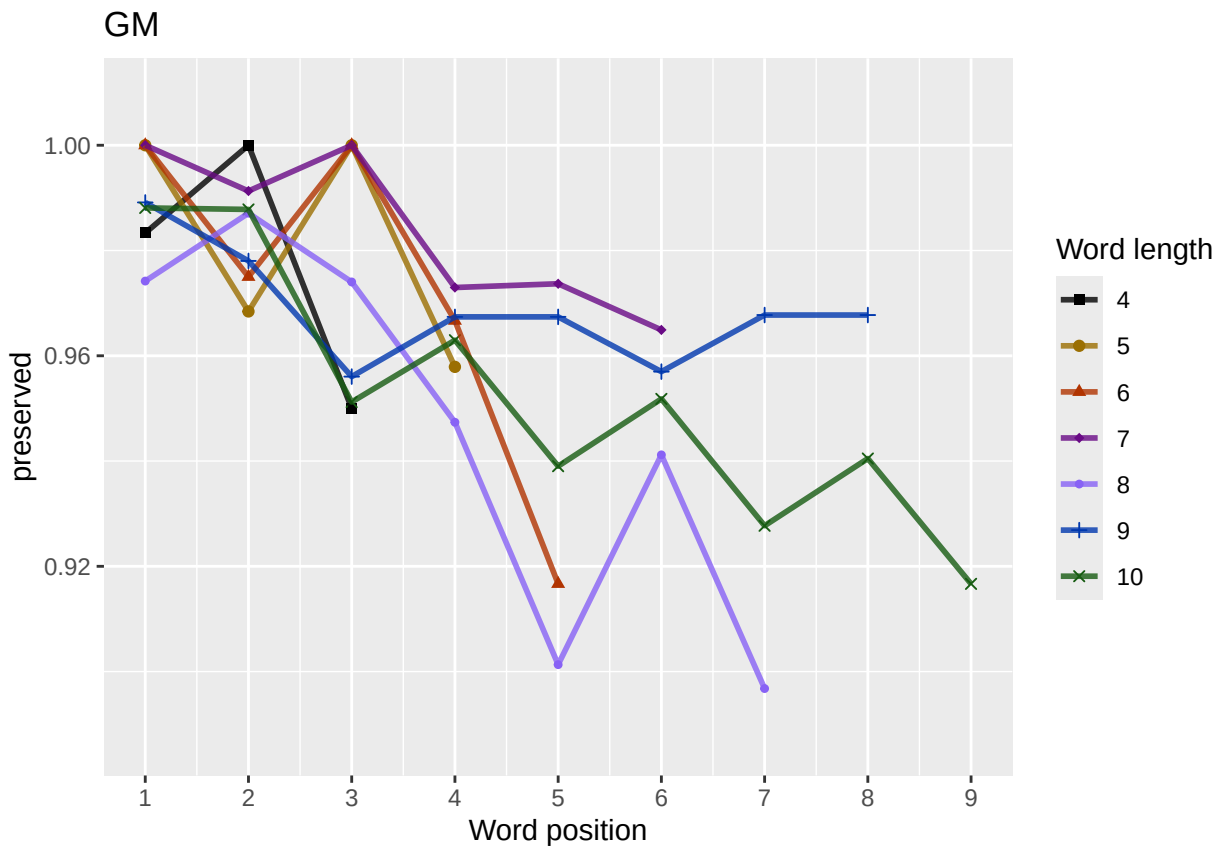
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##     <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1       4    60    59    60    NA    NA    NA    NA    NA    NA
## 2       5    95    95    95    95    NA    NA    NA    NA    NA
## 3       6   120   120   120   120   120    NA    NA    NA    NA
## 4       7   115   115   112   111   114   114    NA    NA    NA
## 5       8   155   155   154   152   152   153   155    NA    NA
## 6       9    92    91    91    92    92    93    93    93    NA
## 7      10    84    82    82    81    82    83    83    84    84
```

```
obs_linetypes <- c("solid","solid","solid","solid",
                   "solid","solid","solid","solid","solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen<-factor(pos_len_summary$stimlen)
pos_len_summary$pos<-factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimle
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                           paste0(CurPat),
                                           NULL,
                                           c(min_preserved,max_preserved),
                                           palette_values,
                                           shape_values,
                                           obs_linetypes,
                                           pred_linetypes)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot,device="png",unit="cm",width=15,height=11)
len_pos_plot
```



Length and position

```
# length and position

LPModelEquations<-c("preserved ~ 1",
            "preserved ~ stimlen",
            "preserved ~ pos",
            "preserved ~ stimlen + pos",
            "preserved ~ stimlen*pos",
            "preserved ~ I(pos^2)+pos",
            "preserved ~ stimlen + I(pos^2) + pos",
            "preserved ~ stimlen * (I(pos^2) + pos)"
)

LPRes<-TestModels(LPModelEquations,PosDat)
```

```
## ****************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     5.39449      0.04961     -0.74107
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1292  AIC: 1298
## log likelihood:  -646.073
## Nagelkerke R2:  0.04795462
## % pres/err predicted correctly:  -296.3271
## % of predictable range [ (model-null)/(1-null) ]:  0.01198305
## ***************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     5.68266     -0.04017      0.05160     -0.74564
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       1348
## Residual Deviance: 1292  AIC: 1300
## log likelihood:  -645.8565
## Nagelkerke R2:  0.04832279
## % pres/err predicted correctly:  -296.3231
## % of predictable range [ (model-null)/(1-null) ]:  0.01199618
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     7.13331     -0.32402     -0.93412      0.07884
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       1348
## Residual Deviance: 1292  AIC: 1300
## log likelihood:  -646.0994
## Nagelkerke R2:  0.04790971
## % pres/err predicted correctly:  -296.2368
## % of predictable range [ (model-null)/(1-null) ]:  0.01228299
## ***************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
```

```
##          7.84193          -0.33400           0.09531          -1.50672          -0.00688           0
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       1348
## Residual Deviance: 1290  AIC: 1302
## log likelihood:  -644.8536
## Nagelkerke R2:  0.05002787
## % pres/err predicted correctly:  -296.2006
## % of predictable range [ (model-null)/(1-null) ]:  0.0124033
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      4.4174      -0.2535
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1300  AIC: 1304
## log likelihood:  -650.1808
## Nagelkerke R2:  0.04096228
## % pres/err predicted correctly:  -296.6452
## % of predictable range [ (model-null)/(1-null) ]:  0.01092596
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    4.466275    -0.007897    -0.250585
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1300  AIC: 1306
## log likelihood:  -650.1721
## Nagelkerke R2:  0.04097699
## % pres/err predicted correctly:  -296.6463
## % of predictable range [ (model-null)/(1-null) ]:  0.01092212
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.5900      -0.1636
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
```

```
## Null Deviance:       1348
## Residual Deviance: 1338  AIC: 1342
## log likelihood:  -668.7569
## Nagelkerke R2:  0.009177715
## % pres/err predicted correctly:  -299.2348
## % of predictable range [ (model-null)/(1-null) ]:  0.002320514
## ****************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       3.303
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4397 Residual
## Null Deviance:       1348
## Residual Deviance: 1348  AIC: 1350
## log likelihood:  -674.0917
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -299.9332
## % of predictable range [ (model-null)/(1-null) ]:  0
## ****************************
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                         AIC=LPRes$AIC,
                         row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)
kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ I(pos^2) + pos	1298.146	0.000000	1.000000	0.4890155	0.0479546	3.394490	NA	-0.7410726	NA	0.0496100	NA
preserved ~ stimlen + I(pos^2) + pos	1299.713	1.567005	0.456808	0.2233388	0.0483228	3.682656	-0.0401709	-0.7456386	NA	0.0516002	NA
preserved ~ stimlen * pos	1300.192	2.052823	0.358290	0.1752096	0.0479097	3.713309	-0.3240162	-0.9341164	0.0788401	NA	NA
preserved ~ stimlen * (I(pos^2) + pos)	1301.707	3.561140	0.168542	0.0824197	0.0500279	3.841931	-0.3340005	-1.5067184	0.1069637	0.0953144	-0.0068798

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ pos	1304.362	2.215485	0.1094701	0.0218599	0.0409624	3.3417355	NA	-0.2535385	NA	NA	NA
preserved ~ stimlen + pos	1306.348	4.198216	0.0403977	0.0081105	0.0409774	0.0466275	-0.0078971	-0.2505849	NA	NA	NA
preserved ~ stimlen	1341.514	43.367784	0.0000000	0.0000000	0.0091774	7590042	-0.1636135	NA	NA	NA	NA
preserved ~ 1	1350.185	52.037283	0.0000000	0.0000000	0.0000000	0.3302934	NA	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ I(pos^2) + pos"
```

```
print(BestLPModel)
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     5.39449      0.04961     -0.74107
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1292  AIC: 1298
```

```
PosDat$LPFitted<-fitted(BestLPModel)
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
                        NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
fitted_pos_len_table
```

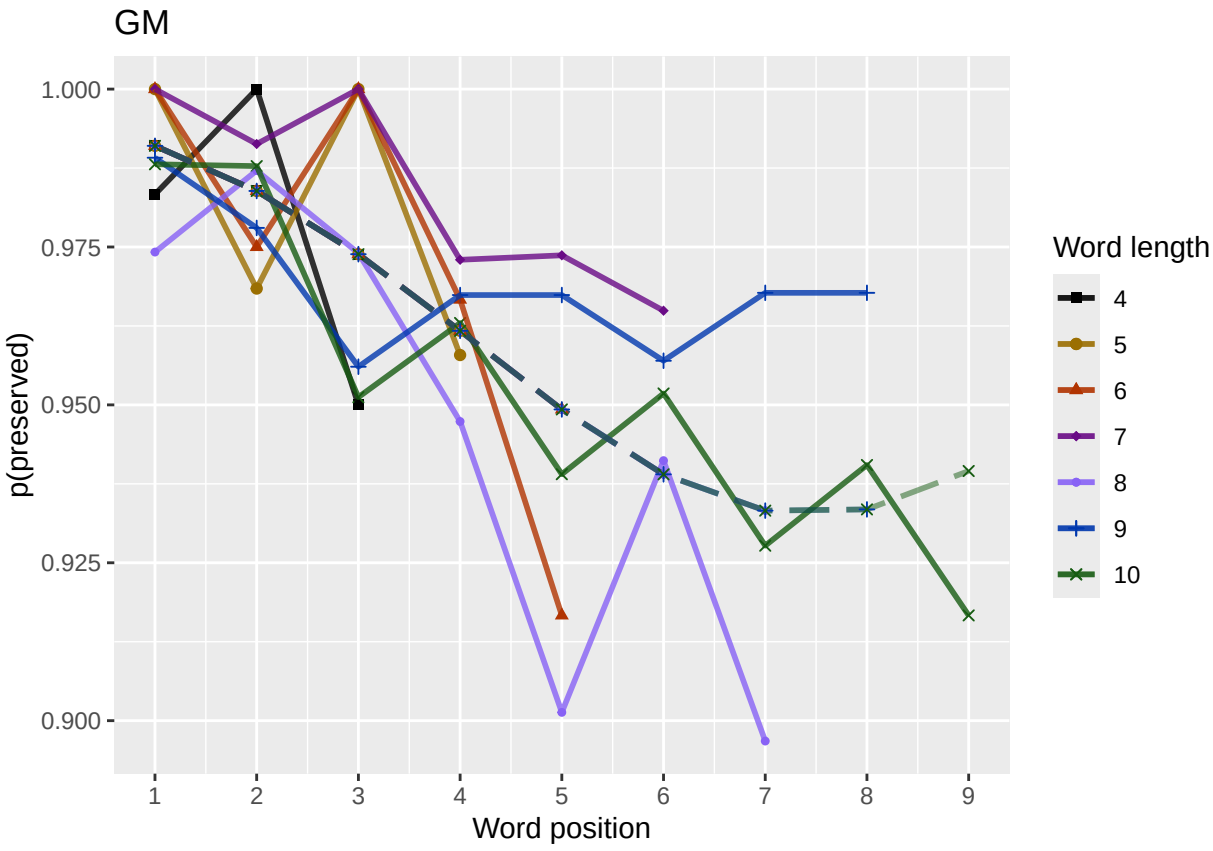
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##     <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1       4 0.991 0.984 0.974 NA     NA     NA     NA     NA     NA
## 2       5 0.991 0.984 0.974  0.962 NA     NA     NA     NA     NA
## 3       6 0.991 0.984 0.974  0.962  0.949 NA     NA     NA     NA
## 4       7 0.991 0.984 0.974  0.962  0.949  0.939 NA     NA     NA
## 5       8 0.991 0.984 0.974  0.962  0.949  0.939  0.933 NA     NA
## 6       9 0.991 0.984 0.974  0.962  0.949  0.939  0.933  0.933 NA
## 7      10 0.991 0.984 0.974  0.962  0.949  0.939  0.933  0.933  0.940
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

```
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitte
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                          paste0(PosDat$patient[1]),
                                          "LPFitted",
                                          NULL,
                                          palette_values,
                                          shape_values,
                                          obs_linetypes,
                                          pred_linetypes = c("longdash")
                                          )
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_percent_preserved_by_length_pos_wfit.png"),plot=
fitted_len_pos_plot
```



length and position without fragments to see if this changes position^2 influence

```
# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num<-0
```

```
prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array
```

```
# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag
```

```
## # A tibble: 1 x 2
##   frag_sum     N
##      <int> <int>
## 1        7   722
```

```
num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100
```

```
## Warning: Unknown or uninitialised column: `percent_with_frag`.
```

```
print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
              num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))
```

```
## [1] "The number of responses with fragments was 7 / 722 = 0.97 percent"
```

```
write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_percent_fragments.csv"),row.names = FALSE)
```

```
# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)
```

```
## ****************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##      5.4609       0.0616      -0.7948
##
## Degrees of Freedom: 4377 Total (i.e. Null);  4375 Residual
## Null Deviance:       1212
## Residual Deviance: 1174  AIC: 1180
## log likelihood:  -586.9553
## Nagelkerke R2:  0.03584981
## % pres/err predicted correctly:  -260.4305
## % of predictable range [ (model-null)/(1-null) ]:  0.008043772
## ****************************
```

```
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     5.71911     -0.03592      0.06344     -0.79930
##
## Degrees of Freedom: 4377 Total (i.e. Null);  4374 Residual
## Null Deviance:       1212
## Residual Deviance: 1174  AIC: 1182
## log likelihood:  -586.7928
## Nagelkerke R2:  0.03615412
## % pres/err predicted correctly:  -260.4282
## % of predictable range [ (model-null)/(1-null) ]:  0.008052608
## **************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     7.40207     -0.36705     -1.02441      0.09507
##
## Degrees of Freedom: 4377 Total (i.e. Null);  4374 Residual
## Null Deviance:       1212
## Residual Deviance: 1174  AIC: 1182
## log likelihood:  -587.0951
## Nagelkerke R2:  0.03558796
## % pres/err predicted correctly:  -260.3706
## % of predictable range [ (model-null)/(1-null) ]:  0.008271047
## **************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##        7.3056199        -0.2720829         0.0479284        -1.1584418        -0.0007051         0.0
##
## Degrees of Freedom: 4377 Total (i.e. Null);  4372 Residual
## Null Deviance:       1212
## Residual Deviance: 1171  AIC: 1183
## log likelihood:  -585.5881
## Nagelkerke R2:  0.0384093
## % pres/err predicted correctly:  -260.2612
## % of predictable range [ (model-null)/(1-null) ]:  0.008686231
## **************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      4.3221      -0.2062
##
## Degrees of Freedom: 4377 Total (i.e. Null);  4376 Residual
## Null Deviance:       1212
## Residual Deviance: 1184  AIC: 1188
## log likelihood:  -592.2369
## Nagelkerke R2:  0.02594699
## % pres/err predicted correctly:  -260.9761
## % of predictable range [ (model-null)/(1-null) ]:  0.005973666
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##   4.3168430    0.0008461   -0.2064834
##
## Degrees of Freedom: 4377 Total (i.e. Null);  4375 Residual
## Null Deviance:       1212
## Residual Deviance: 1184  AIC: 1190
## log likelihood:  -592.2368
## Nagelkerke R2:  0.02594716
## % pres/err predicted correctly:  -260.9758
## % of predictable range [ (model-null)/(1-null) ]:  0.005974933
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.3981      -0.1226
##
## Degrees of Freedom: 4377 Total (i.e. Null);  4376 Residual
## Null Deviance:       1212
## Residual Deviance: 1207  AIC: 1211
## log likelihood:  -603.3317
## Nagelkerke R2:  0.005066852
## % pres/err predicted correctly:  -262.2407
## % of predictable range [ (model-null)/(1-null) ]:  0.001175365
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
```

```
## (Intercept)
##        3.44
##
## Degrees of Freedom: 4377 Total (i.e. Null);  4377 Residual
## Null Deviance:       1212
## Residual Deviance: 1212  AIC: 1214
## log likelihood:  -606.0156
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -262.5505
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                       AIC=NoFrag_LPRes$AIC,
                       row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ I(pos^2) + pos	1179.91	0.000000	1.000000	0.5084007	0.7035849	3.460893	NA	-0.7947518	NA	0.0616035	NA
preserved ~ stimlen + I(pos^2) + pos	1181.585	1.674988	0.4327938	0.2200326	0.7036154	1.719115	-0.0359219	-0.7992954	NA	0.0634399	NA
preserved ~ stimlen * pos	1182.192	2.279637	0.3198771	0.1626257	0.7035587	0.402065	-0.3670504	-0.0244122	0.0950658	NA	NA
preserved ~ stimlen * (I(pos^2) + pos)	1183.178	3.265647	0.1953771	0.0993299	0.7038407	3.305620	-0.2720829	-1.1584418	0.0657619	0.0479284	-0.0007051
preserved ~ pos	1188.478	4.563303	0.0138198	0.0070260	0.0259470	3.322118	NA	-0.2061752	NA	NA	NA
preserved ~ stimlen + pos	1190.474	10.563108	0.0050845	0.0025850	0.0259472	3.168430	-0.0008461	-0.2064834	NA	NA	NA
preserved ~ stimlen	1210.663	30.752902	0.0000000	0.0000000	0.0100506	3.398104	-0.1226289	NA	NA	NA	NA
preserved ~ 1	1214.033	34.120597	0.0000000	0.0000000	0.0000000	3.440135	NA	NA	NA	NA	NA

```
# plot no fragment data

NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
                          NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

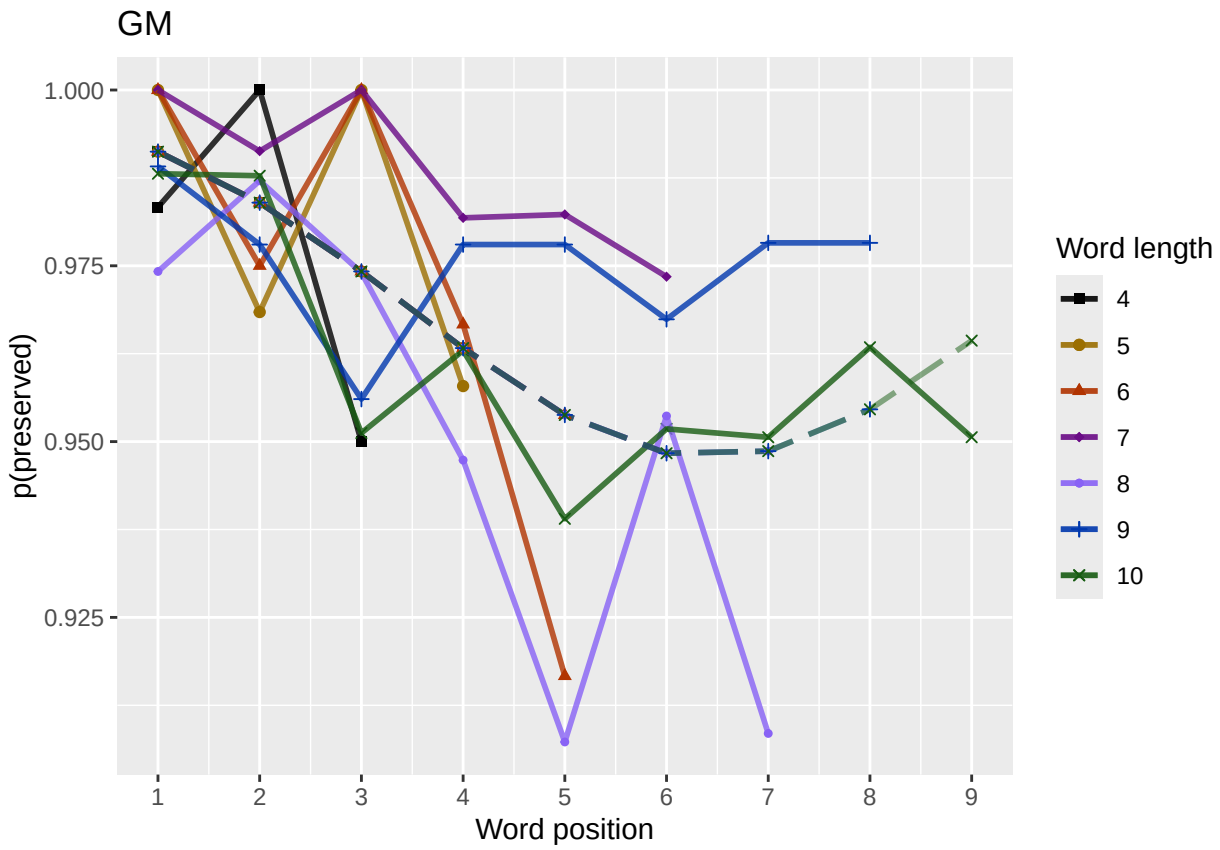
```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.991 0.984 0.974 NA     NA     NA     NA     NA     NA
## 2       5 0.991 0.984 0.974  0.963 NA     NA     NA     NA     NA
## 3       6 0.991 0.984 0.974  0.963  0.954 NA     NA     NA     NA
## 4       7 0.991 0.984 0.974  0.963  0.954  0.948 NA     NA     NA
## 5       8 0.991 0.984 0.974  0.963  0.954  0.948  0.949 NA     NA
## 6       9 0.991 0.984 0.974  0.963  0.954  0.948  0.949  0.955 NA
## 7      10 0.991 0.984 0.974  0.963  0.954  0.948  0.949  0.955  0.964
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitte
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
                                        paste0(NoFragData$patient[1]),
                                        "LPFitted",
                                        NULL,
                                        palette_values,
                                        shape_values,
                                        obs_linetypes,
                                        pred_linetypes = c("longdash")
                                        )
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
              "no_fragments_percent_preserved_by_length_pos_wfit.png"),
       plot=nofrag_fitted_len_pos_plot,
       device="png",unit="cm",
       width=15,height=11)
nofrag_fitted_len_pos_plot
```



back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.89 - 1.01"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account

# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]

# take averages for positions
# don't want downward estimates influenced by return upward of U
# therefore, for downward influence, use only the values before the min
# take the difference between each value (differences between position proportion correct) **NOTE** pro
# average the difference in probabilities

# in case min is close to the end or we are not using a min (for non-U-shaped)
# functions, we need to get differences _first_ and then average those (e.g.
# if there is an effect of length and we just average position data,
# later positions) will have a lower average (because of the length effect)
# even if all position functions go upward
```

```
table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)
```

```
## [1] "mean change in probability for each additional length: "
```

```
print(OA_mean_len_diff)
```

```
## [1] 0
```

```
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding  U-shape): "
print(CurrentLabel)
```

```
## [1] "mean change in probability for each additional position (excluding  U-shape): "
```

```
print(OA_mean_pos_diff)
```

```
## [1] -0.009625712
```

```
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)
```

```
if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                               2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
```

```
    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}
```

```
## [1] "Average upward change after U minimum"
## [1] 0.00313561
```

```
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upwa
```

```
  CurrentLabel<-"return upward value"
  print(upward_dist[biggest_return_upward_row])
   results_report_DF <- AddReportLine(results_report_DF,
                                      CurrentLabel,
                                      upward_dist[biggest_return_upward_row])

  print(" ")
  # percentage return upward
  percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upwar
  CurrentLabel <- "Return upward as a proportion of the downward distance:"
  print(CurrentLabel)
  print(percentage_return_upward)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                         percentage_return_upward)
}else{
  print("no U-shape in this participant")
}
```

```
## [1] "differences from left max to min for each row: "
## [1] 0.01715469 0.02928768 0.04173405 0.05200960 0.05775427 0.05775427 0.05775427
## [1] "differences from min to right max for each row: "
## [1] 0.0000000000 0.0000000000 0.0000000000 0.0000000000 0.0000000000 0.0001913896 0.0062712194
## [1] " "
## [1] "row with biggest return upward"
## [1] 7
## [1] " "
## [1] "downward distance for row with the largest upward value"
## [1] 0.05775427
## [1] 0.006271219
## [1] " "
## [1] "Return upward as a proportion of the downward distance:"
## [1] 0.1085845
```

```
FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
            "preserved ~ stimlen*log_freq",
            "preserved ~ stimlen+log_freq",
            "preserved ~ pos*log_freq",
            "preserved ~ pos+log_freq",
            "preserved ~ stimlen*log_freq + pos*log_freq",
            "preserved ~ stimlen*log_freq + pos",
            "preserved ~ stimlen + pos*log_freq",
            "preserved ~ stimlen + pos + log_freq",
            "preserved ~ (I(pos^2)+pos)*log_freq",
            "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
            "preserved ~ stimlen*log_freq + I(pos^2) + pos",
            "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
            "preserved ~ stimlen + I(pos^2) + pos + log_freq",

            # models without frequency
            "preserved ~ 1",
            "preserved ~ stimlen",
            "preserved ~ pos",
            "preserved ~ stimlen + pos",
```

```
            "preserved ~ stimlen*pos",
            "preserved ~ I(pos^2)+pos",
            "preserved ~ stimlen + I(pos^2) + pos",
            "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)
```

```
## ***************************
## model index:  9
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)           I(pos^2)                pos           log_freq  I(pos^2):log_freq
##           5.56515            0.06375           -0.84642            0.33795            0.01617
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       1348
## Residual Deviance: 1283  AIC: 1295
## log likelihood:  -641.4067
## Nagelkerke R2:  0.05588196
## % pres/err predicted correctly:  -295.4041
## % of predictable range [ (model-null)/(1-null) ]:  0.01505014
## ***************************
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos     log_freq
##    5.447857    -0.008464     0.051223    -0.741867     0.103065
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4393 Residual
## Null Deviance:       1348
## Residual Deviance: 1286  AIC: 1296
## log likelihood:  -643.18
## Nagelkerke R2:  0.05287131
## % pres/err predicted correctly:  -295.7561
## % of predictable range [ (model-null)/(1-null) ]:  0.01388033
## ***************************
## model index:  12
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)           stimlen           I(pos^2)                pos           log_freq  I(pos
##           5.69934           -0.01848            0.06475           -0.84941            0.33137
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4391 Residual
## Null Deviance:       1348
```

```
## Residual Deviance: 1283  AIC: 1297
## log likelihood:  -641.3643
## Nagelkerke R2:  0.05595393
## % pres/err predicted correctly:  -295.4122
## % of predictable range [ (model-null)/(1-null) ]:  0.01502308
## ***************************
## model index:  10
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)            stimlen           log_freq           I(pos^2)                pos   stim
##           5.69281           -0.02248            0.59514            0.06545           -0.84876
##      log_freq:pos
##          -0.14632
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4390 Residual
## Null Deviance:       1348
## Residual Deviance: 1282  AIC: 1298
## log likelihood:  -640.7704
## Nagelkerke R2:  0.05696156
## % pres/err predicted correctly:  -295.2926
## % of predictable range [ (model-null)/(1-null) ]:  0.0154204
## ***************************
## model index:  19
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     5.39449      0.04961     -0.74107
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1292  AIC: 1298
## log likelihood:  -646.073
## Nagelkerke R2:  0.04795462
## % pres/err predicted correctly:  -296.3271
## % of predictable range [ (model-null)/(1-null) ]:  0.01198305
## ***************************
## model index:  11
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq          I(pos^2)               pos  stimlen:log
##         5.438234         -0.008833          0.166965          0.050875         -0.738412         -0.0
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       1348
## Residual Deviance: 1286  AIC: 1298
```

```
## log likelihood:  -643.1411
## Nagelkerke R2:  0.05293752
## % pres/err predicted correctly:  -295.7584
## % of predictable range [ (model-null)/(1-null) ]:  0.01387263
## **************************
## model index:  20
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     5.68266     -0.04017      0.05160     -0.74564
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       1348
## Residual Deviance: 1292  AIC: 1300
## log likelihood:  -645.8565
## Nagelkerke R2:  0.04832279
## % pres/err predicted correctly:  -296.3231
## % of predictable range [ (model-null)/(1-null) ]:  0.01199618
## **************************
## model index:  18
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     7.13331     -0.32402     -0.93412      0.07884
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       1348
## Residual Deviance: 1292  AIC: 1300
## log likelihood:  -646.0994
## Nagelkerke R2:  0.04790971
## % pres/err predicted correctly:  -296.2368
## % of predictable range [ (model-null)/(1-null) ]:  0.01228299
## **************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos     log_freq
##      4.3901      -0.2425       0.1007
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1295  AIC: 1301
## log likelihood:  -647.4905
## Nagelkerke R2:  0.04554312
## % pres/err predicted correctly:  -296.0414
```

```
## % of predictable range [ (model-null)/(1-null) ]:  0.0129322
## ***************************
## model index:  21
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stimle
##          7.84193          -0.33400           0.09531          -1.50672          -0.00688            0
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       1348
## Residual Deviance: 1290  AIC: 1302
## log likelihood:  -644.8536
## Nagelkerke R2:  0.05002787
## % pres/err predicted correctly:  -296.2006
## % of predictable range [ (model-null)/(1-null) ]:  0.0124033
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##  (Intercept)           pos      log_freq  pos:log_freq
##     4.374756     -0.238305      0.057418      0.008817
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       1348
## Residual Deviance: 1295  AIC: 1303
## log likelihood:  -647.385
## Nagelkerke R2:  0.04572269
## % pres/err predicted correctly:  -295.9488
## % of predictable range [ (model-null)/(1-null) ]:  0.01323994
## ***************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos     log_freq
##     4.23780      0.02423     -0.25090      0.10455
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       1348
## Residual Deviance: 1295  AIC: 1303
## log likelihood:  -647.4141
## Nagelkerke R2:  0.04567329
## % pres/err predicted correctly:  -296.0097
## % of predictable range [ (model-null)/(1-null) ]:  0.01303768
## ***************************
## model index:  16
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      4.4174      -0.2535
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1300  AIC: 1304
## log likelihood:  -650.1808
## Nagelkerke R2:  0.04096228
## % pres/err predicted correctly:  -296.6452
## % of predictable range [ (model-null)/(1-null) ]:  0.01092596
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq               pos  stimlen:log_freq
##          4.23450           0.02352           0.21255          -0.25100          -0.01337
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4393 Residual
## Null Deviance:       1348
## Residual Deviance: 1295  AIC: 1305
## log likelihood:  -647.2943
## Nagelkerke R2:  0.04587711
## % pres/err predicted correctly:  -296.035
## % of predictable range [ (model-null)/(1-null) ]:  0.01295357
## ***************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##  (Intercept)       stimlen           pos      log_freq  pos:log_freq
##     4.244559      0.020917     -0.245923      0.064747      0.008006
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4393 Residual
## Null Deviance:       1348
## Residual Deviance: 1295  AIC: 1305
## log likelihood:  -647.3289
## Nagelkerke R2:  0.04581816
## % pres/err predicted correctly:  -295.9306
## % of predictable range [ (model-null)/(1-null) ]:  0.0133006
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
##      (Intercept)           stimlen          log_freq               pos  stimlen:log_freq      log_fr
##          4.24010           0.01683           0.22958          -0.24096          -0.02653           0
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       1348
## Residual Deviance: 1294  AIC: 1306
## log likelihood:  -646.9864
## Nagelkerke R2:  0.04640102
## % pres/err predicted correctly:  -295.877
## % of predictable range [ (model-null)/(1-null) ]:  0.01347866
## ***************************
## model index:  17
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    4.466275    -0.007897    -0.250585
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1300  AIC: 1306
## log likelihood:  -650.1721
## Nagelkerke R2:  0.04097699
## % pres/err predicted correctly:  -296.6463
## % of predictable range [ (model-null)/(1-null) ]:  0.01092212
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     log_freq
##      4.3644      -0.1321       0.1035
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1332  AIC: 1338
## log likelihood:  -666.0219
## Nagelkerke R2:  0.01387432
## % pres/err predicted correctly:  -298.82
## % of predictable range [ (model-null)/(1-null) ]:  0.003699109
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq  stimlen:log_freq
```

```
##          4.36154          -0.13282           0.20693          -0.01278
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       1348
## Residual Deviance: 1332  AIC: 1340
## log likelihood:  -665.9119
## Nagelkerke R2:  0.0140631
## % pres/err predicted correctly:  -298.8209
## % of predictable range [ (model-null)/(1-null) ]:  0.003696101
## ***************************
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.5900      -0.1636
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1338  AIC: 1342
## log likelihood:  -668.7569
## Nagelkerke R2:  0.009177715
## % pres/err predicted correctly:  -299.2348
## % of predictable range [ (model-null)/(1-null) ]:  0.002320514
## ***************************
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       3.303
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4397 Residual
## Null Deviance:       1348
## Residual Deviance: 1348  AIC: 1350
## log likelihood:  -674.0917
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -299.9332
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2
```

```
FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                       by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_freq_len_pos_model_summary.csv"),row.names = FALSE)
kable(FLPAICSummary)
```

Model	AIC	DeltaAIC	AICExp	AICw	NagR2	(Intercept)	stimlen	log_freq	stimlen:log_freq	pos	pos:log_freq	log_freq:pos	I(pos^2)	I(pos^2):log_freq	log_freq:I(pos^2)	stimlen:pos	stimlen:I(pos^2)
preserved ~ (I(pos^2) + pos) * log_freq	1294.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	NA	[illegible]	NA	[illegible]	[illegible]	NA	[illegible]	[illegible]	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos + log_freq	1296.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	-0.0084639	[illegible]	NA	-0.7418673	NA	NA	[illegible]	NA	NA	NA	NA
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	1296.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	-0.0184770	[illegible]	NA	[illegible]	[illegible]	NA	[illegible]	[illegible]	NA	NA	NA
preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq	1297.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	-0.0224820	0.5951443	[illegible]	[illegible]	NA	-0.1463213	[illegible]	NA	[illegible]	NA	NA
preserved ~ I(pos^2) + pos	1298.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	NA	NA	NA	-0.7410726	NA	NA	[illegible]	NA	NA	NA	NA
preserved ~ stimlen * log_freq + I(pos^2) + pos	1298.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	-0.0088327	0.1669653	[illegible]	[illegible]	NA	NA	[illegible]	NA	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos	1299.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	-0.0401709	NA	NA	-0.7456386	NA	NA	[illegible]	NA	NA	NA	NA
preserved ~ stimlen * pos	1300.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	-0.3240162	NA	NA	-0.9341164	NA	NA	NA	NA	NA	[illegible]	NA
preserved ~ pos + log_freq	1300.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	NA	[illegible]	NA	-0.2425029	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * (I(pos^2) + pos)	1301.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	-0.3340005	NA	NA	-1.5067184	NA	NA	[illegible]	NA	NA	0.1069637	-0.0068798

Model	AIC	DeltaAIC	AICwt	NagR2	(Intercept)	stimlen	log_freq	stimlen:log_freq	pos	log_freq:pos	I(pos^2)	log_freq:I(pos^2)	pos:I(pos^2)	log_freq:pos:I(pos^2)
preserved ~ pos * log_freq	1302.7	7.7	0.0187	0.0857	0.0574	NA	NA	NA	-0.0088	NA	NA	NA	NA	NA
preserved ~ stimlen + pos + log_freq	1302.8	7.8	0.0180	0.0857	0.2383047	NA	NA	NA	-0.2509036	NA	NA	NA	NA	NA
preserved ~ pos	1304.3	9.3	0.0083	0.0809	0.2535385	NA	NA	NA	-0.2459230	NA	NA	NA	NA	NA
preserved ~ stimlen * log_freq + pos	1304.9	9.9	0.0061	0.0799	0.0133662	0.2509955	-0.2505849	NA	-0.0078971	NA	NA	NA	NA	NA
preserved ~ stimlen + pos * log_freq	1304.9	9.9	0.0061	0.0799	0.2459230	NA	NA	NA	-0.0080179	NA	NA	NA	NA	NA
preserved ~ stimlen * log_freq + pos * log_freq	1305.1	10.1	0.0055	0.0795	0.0265298	0.2409628	-0.2320826	-0.0168322	-0.0179557	NA	NA	NA	NA	NA
preserved ~ stimlen + pos	1306.3	11.3	0.0030	0.0776	0.2505849	NA	NA	NA	-0.0078971	NA	NA	NA	NA	NA
preserved ~ stimlen + log_freq	1338.4	43.4	0.0000	0.0363	0.1320580	0.1035246	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * log_freq	1339.4	44.4	0.0000	0.0346	0.1328159	0.2069339	0.0127835	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen	1341.4	46.4	0.0000	0.0317	0.1636135	0.1770042	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ 1	1350.5	55.6	0.0000	0.0183	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ (I(pos^2) + pos) * log_freq"
```

```
print(BestFLPModel)
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)          I(pos^2)               pos          log_freq  I(pos^2):log_freq
##           5.56515            0.06375           -0.84642            0.33795            0.01617
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       1348
## Residual Deviance: 1283  AIC: 1295
```

```
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq]<-"hf"
```

```
PosDat$freq_bin[PosDat$log_freq <  median_freq]<-"lf"
```

```
PosDat$FLPFitted<-fitted(BestFLPModel)

HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]

HF_Plot <- plot_len_pos_obs_predicted(HFDat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitte
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
LF_Plot <- plot_len_pos_obs_predicted(LFDat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

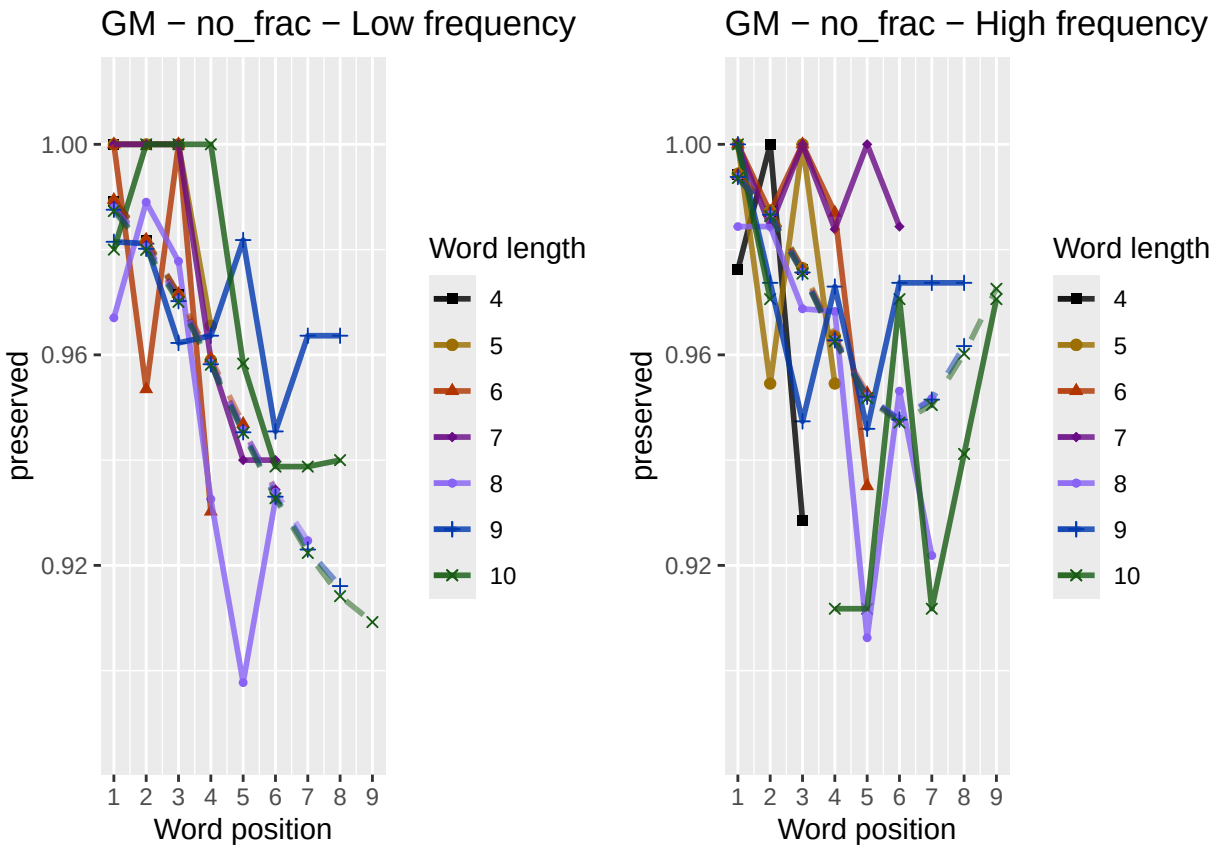
```
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`)
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)
```




```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## ****************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.702       -1.760
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1128  AIC: 1132
## log likelihood:  -564.2311
## Nagelkerke R2:  0.1845804
## % pres/err predicted correctly:  -255.0358
```

```
## % of predictable range [ (model-null)/(1-null) ]:  0.1491937
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     5.39449      0.04961     -0.74107
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1292  AIC: 1298
## log likelihood:  -646.073
## Nagelkerke R2:  0.04795462
## % pres/err predicted correctly:  -296.3271
## % of predictable range [ (model-null)/(1-null) ]:  0.01198305
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      4.4174      -0.2535
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1300  AIC: 1304
## log likelihood:  -650.1808
## Nagelkerke R2:  0.04096228
## % pres/err predicted correctly:  -296.6452
## % of predictable range [ (model-null)/(1-null) ]:  0.01092596
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.5900      -0.1636
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1338  AIC: 1342
## log likelihood:  -668.7569
## Nagelkerke R2:  0.009177715
## % pres/err predicted correctly:  -299.2348
## % of predictable range [ (model-null)/(1-null) ]:  0.002320514
## ***************************
## model index:  1
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      3.5165      -0.0739
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1344  AIC: 1348
## log likelihood:  -672.2345
## Nagelkerke R2:  0.00319751
## % pres/err predicted correctly:  -299.7032
## % of predictable range [ (model-null)/(1-null) ]:  0.0007639755
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       3.303
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4397 Residual
## Null Deviance:       1348
## Residual Deviance: 1348  AIC: 1350
## log likelihood:  -674.0917
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -299.9332
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                         AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                          by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                              CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(posˆ2)	pos	stimlen
preserved ~ CumErr	1132.462	0.0000	1	1	0.1845803	3.702231	NA	-1.760047	NA	NA	NA
preserved ~ (I(posˆ2) + pos)	1298.146	165.6838	0	0	0.0479546	5.394490	NA	NA	0.04961	-0.7410726	NA
preserved ~ pos	1304.362	171.8992	0	0	0.0409624	4.417355	NA	NA	NA	-0.2535385	NA
preserved ~ stimlen	1341.514	209.0515	0	0	0.0091774	4.590042	NA	NA	NA	NA	-0.1636135
preserved ~ CumPres	1348.469	216.0067	0	0	0.0031973	5.516533	-0.0738972	NA	NA	NA	NA
preserved ~ 1	1350.183	217.7210	0	0	0.0000000	6.302934	NA	NA	NA	NA	NA

```
if(DoSimulations){
      BestMEModelFormulaRnd <- BestMEModelFormula
    if(grepl("CumPres",BestMEModelFormulaRnd)){
      BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
    }else if(grepl("CumErr",BestMEModelFormulaRnd)){
      BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
    }

    RndModelAIC<-numeric(length=RandomSamples)
    for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
      PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
      PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
      BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                          family="binomial",data=PosDat)
      RndModelAIC[rindex] <- BestModelRnd$aic
    }
    ModelNames<-c(paste0("***",BestMEModelFormula),
                  rep(BestMEModelFormulaRnd,RandomSamples))
    AICValues <- c(BestMEModel$aic,RndModelAIC)
    BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
    BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
    BestMEModelRndDF <- rbind(BestMEModelRndDF,
                              data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
    BestMEModelRndDF <- rbind(BestMEModelRndDF,
                              data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

    write.csv(BestMEModelRndDF,
              paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                     "_best_main_effects_model_with_random_cum_term.csv"),
              row.names = FALSE)
}
```

```
syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                        RunLabel,"_",CurTask,
```

```
                                        "_syllable_component_summary.csv"),row.names = FALSE)
kable(syll_component_summary)
```

syll_component	MeanPres	N
1	0.9740484	578
O	0.9532250	2031
P	1.0000000	36
S	0.9570312	256
V	0.9766199	1497

```
#  main effects models for data without satellite positions

keep_components = c("O","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                              stimlen,stim,pos,
                              preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.673       -1.719
##
## Degrees of Freedom: 4105 Total (i.e. Null);  4104 Residual
## Null Deviance:       1254
## Residual Deviance: 1065  AIC: 1069
## log likelihood:  -532.5692
## Nagelkerke R2:  0.1711416
## % pres/err predicted correctly:  -240.5557
## % of predictable range [ (model-null)/(1-null) ]:  0.1365577
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     5.26815      0.04671     -0.69648
##
## Degrees of Freedom: 4105 Total (i.e. Null);  4103 Residual
## Null Deviance:       1254
## Residual Deviance: 1208  AIC: 1214
```

```
## log likelihood:  -603.9342
## Nagelkerke R2:  0.0428279
## % pres/err predicted correctly:  -275.7933
## % of predictable range [ (model-null)/(1-null) ]:  0.01060057
## **************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      4.3604      -0.2398
##
## Degrees of Freedom: 4105 Total (i.e. Null);  4104 Residual
## Null Deviance:       1254
## Residual Deviance: 1215  AIC: 1219
## log likelihood:  -607.3542
## Nagelkerke R2:  0.03656607
## % pres/err predicted correctly:  -276.0502
## % of predictable range [ (model-null)/(1-null) ]:  0.009682222
## **************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.6477      -0.1701
##
## Degrees of Freedom: 4105 Total (i.e. Null);  4104 Residual
## Null Deviance:       1254
## Residual Deviance: 1244  AIC: 1248
## log likelihood:  -621.8605
## Nagelkerke R2:  0.009889504
## % pres/err predicted correctly:  -278.0503
## % of predictable range [ (model-null)/(1-null) ]:  0.0025329
## **************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##     3.51269     -0.07606
##
## Degrees of Freedom: 4105 Total (i.e. Null);  4104 Residual
## Null Deviance:       1254
## Residual Deviance: 1251  AIC: 1255
## log likelihood:  -625.5355
## Nagelkerke R2:  0.003101225
## % pres/err predicted correctly:  -278.548
```

```
## % of predictable range [ (model-null)/(1-null) ]:  0.0007537717
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       3.308
##
## Degrees of Freedom: 4105 Total (i.e. Null);  4105 Residual
## Null Deviance:       1254
## Residual Deviance: 1254  AIC: 1256
## log likelihood:  -627.2123
## Nagelkerke R2:  -8.434661e-16
## % pres/err predicted correctly:  -278.7589
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
write.csv(SimpSyllMEAICSummary,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",
                 CurTask,
                 "_OV1_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1069.138	0.0000	1	1	0.1711416	3.673081	NA	-1.71915	NA	NA	NA
preserved ~ (I(pos^2) + pos)	1213.868	144.7300	0	0	0.0428279	3.268148	NA	NA	0.0467051	-0.6964764	NA
preserved ~ pos	1218.708	149.5700	0	0	0.0365664	1.360433	NA	NA	NA	-0.2397750	NA
preserved ~ stimlen	1247.721	178.5826	0	0	0.0098895	4.647663	NA	NA	NA	NA	-0.1700981
preserved ~ CumPres	1255.071	185.9327	0	0	0.0031013	2.512686	-0.0760615	NA	NA	NA	NA
preserved ~ 1	1256.425	187.2863	0	0	0.0000000	3.307518	NA	NA	NA	NA	NA

```
#  main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
#  also reduces data)

keep_components = c("O","V")
OVData <- PosDat[PosDat$syll_component %in% keep_components,]
OVData <- OVData %>% select(stim_number,
                            stimlen,stim,pos,
                            preserved,syll_component)
OVData$CumPres <- CalcCumPres(OVData)
OVData$CumErr <- CalcCumErrFromPreserved(OVData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVData,MEModelEquations)
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.557       -1.818
##
## Degrees of Freedom: 3527 Total (i.e. Null);  3526 Residual
## Null Deviance:       1113
## Residual Deviance: 996.6     AIC: 1001
## log likelihood:  -498.3058
## Nagelkerke R2:  0.1203094
## % pres/err predicted correctly:  -228.2346
## % of predictable range [ (model-null)/(1-null) ]:  0.08459783
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     5.28510      0.05067     -0.73340
##
## Degrees of Freedom: 3527 Total (i.e. Null);  3525 Residual
## Null Deviance:       1113
## Residual Deviance: 1068  AIC: 1074
## log likelihood:  -534.1269
## Nagelkerke R2:  0.04697839
## % pres/err predicted correctly:  -246.4695
## % of predictable range [ (model-null)/(1-null) ]:  0.01178005
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      4.3161      -0.2394
##
## Degrees of Freedom: 3527 Total (i.e. Null);  3526 Residual
## Null Deviance:       1113
## Residual Deviance: 1076  AIC: 1080
## log likelihood:  -538.0007
## Nagelkerke R2:  0.03895847
## % pres/err predicted correctly:  -246.8006
## % of predictable range [ (model-null)/(1-null) ]:  0.01045797
## ***************************
## model index:  1
##
```

```
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      3.5676      -0.1307
##
## Degrees of Freedom: 3527 Total (i.e. Null);  3526 Residual
## Null Deviance:       1113
## Residual Deviance: 1107  AIC: 1111
## log likelihood:  -553.2699
## Nagelkerke R2:  0.007175116
## % pres/err predicted correctly:  -248.9544
## % of predictable range [ (model-null)/(1-null) ]:  0.0018574
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.3635      -0.1406
##
## Degrees of Freedom: 3527 Total (i.e. Null);  3526 Residual
## Null Deviance:       1113
## Residual Deviance: 1107  AIC: 1111
## log likelihood:  -553.3303
## Nagelkerke R2:  0.00704871
## % pres/err predicted correctly:  -248.9551
## % of predictable range [ (model-null)/(1-null) ]:  0.00185462
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       3.263
##
## Degrees of Freedom: 3527 Total (i.e. Null);  3527 Residual
## Null Deviance:       1113
## Residual Deviance: 1113  AIC: 1115
## log likelihood:  -556.6987
## Nagelkerke R2:  4.102203e-16
## % pres/err predicted correctly:  -249.4195
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

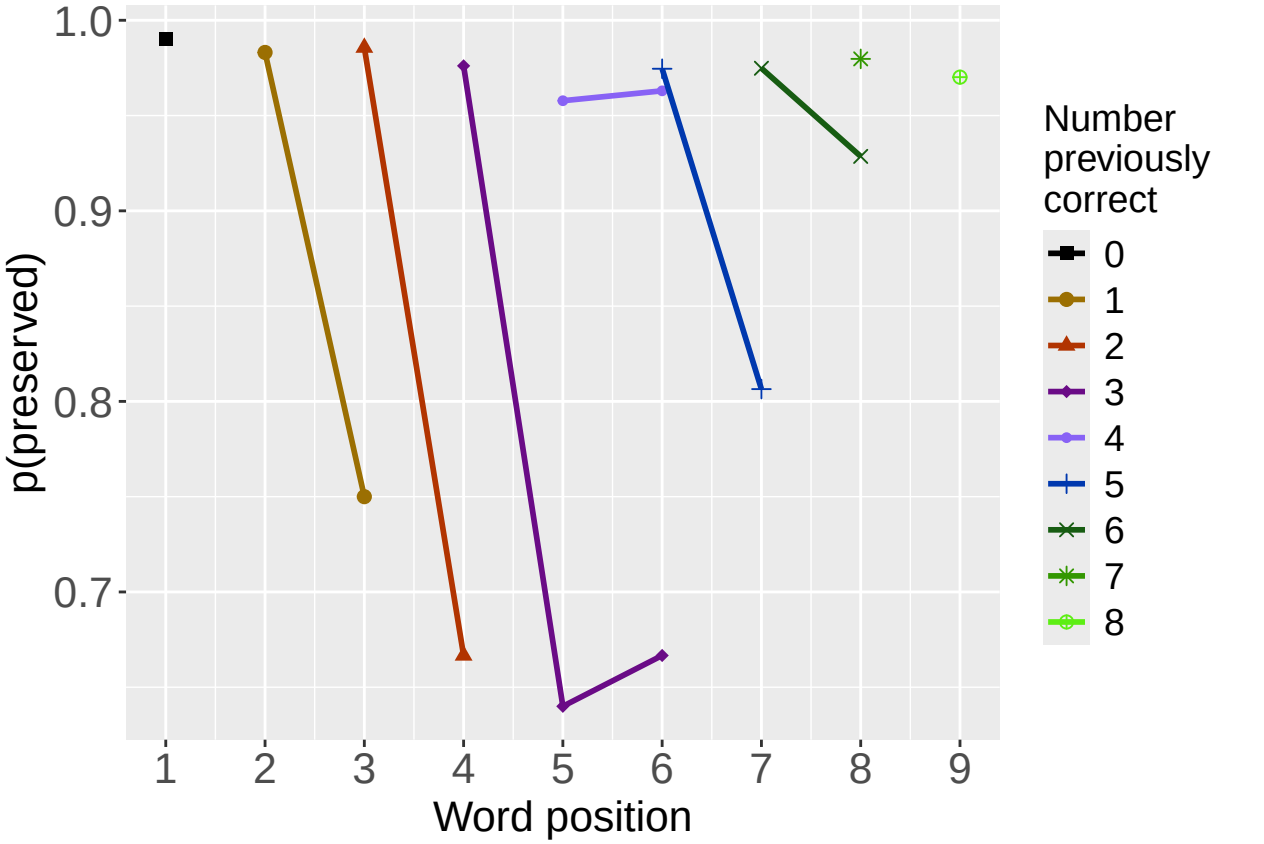
```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                 "_OV_main_effects_model_summary.csv"),row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(posˆ2)	pos	stimlen
preserved ~ CumErr	1000.612	0.00000	1	1	0.1203093	4.557167	NA	-1.817729	NA	NA	NA
preserved ~ (I(posˆ2) + pos)	1074.254	73.64217	0	0	0.0469784	5.285102	NA	NA	0.0506742	-0.7333950	NA
preserved ~ pos	1080.001	79.38985	0	0	0.0389584	5.316080	NA	NA	NA	-0.2394447	NA
preserved ~ CumPres	1110.540	109.92808	0	0	0.0071753	3.567628	-0.1306554	NA	NA	NA	NA
preserved ~ stimlen	1110.661	110.04901	0	0	0.0070487	7.363459	NA	NA	NA	NA	-0.1405966
preserved ~ 1	1115.397	114.78569	0	0	0.0000000	0.263408	NA	NA	NA	NA	NA

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

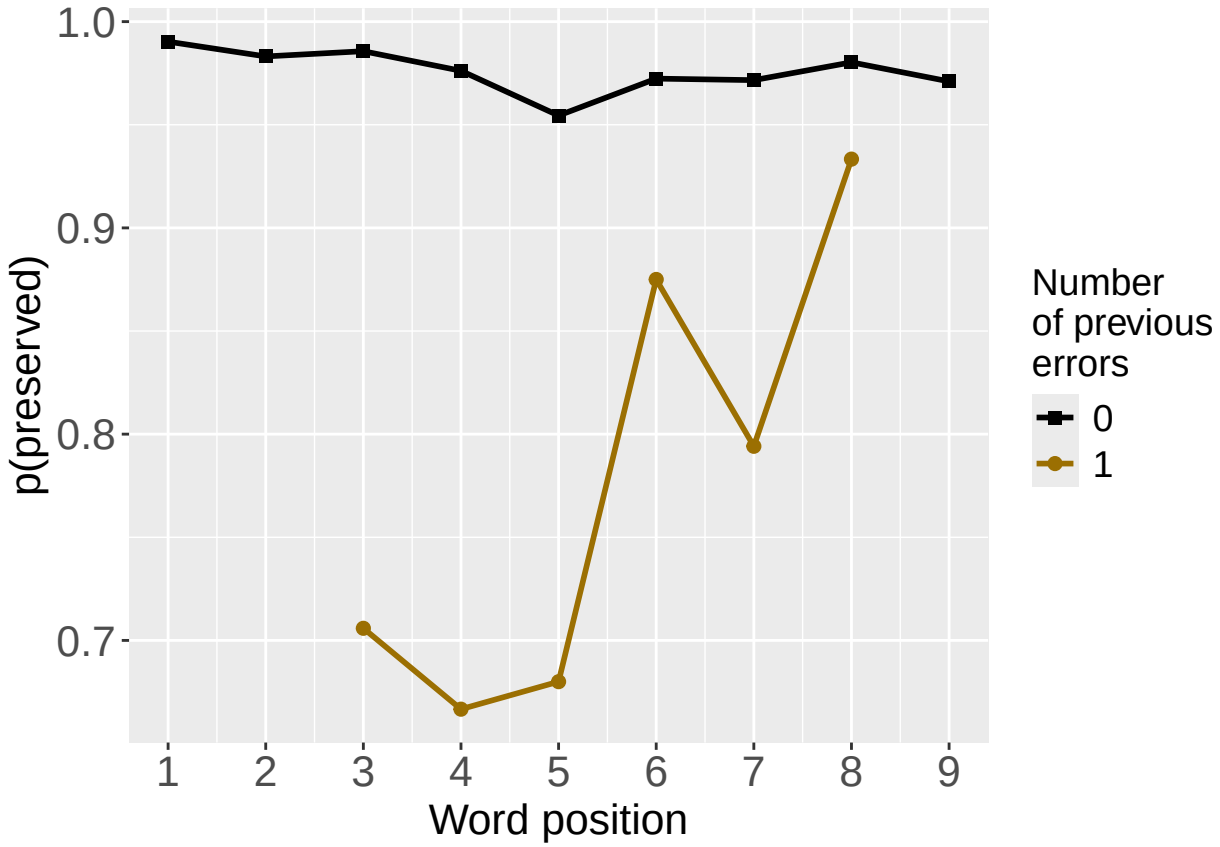
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

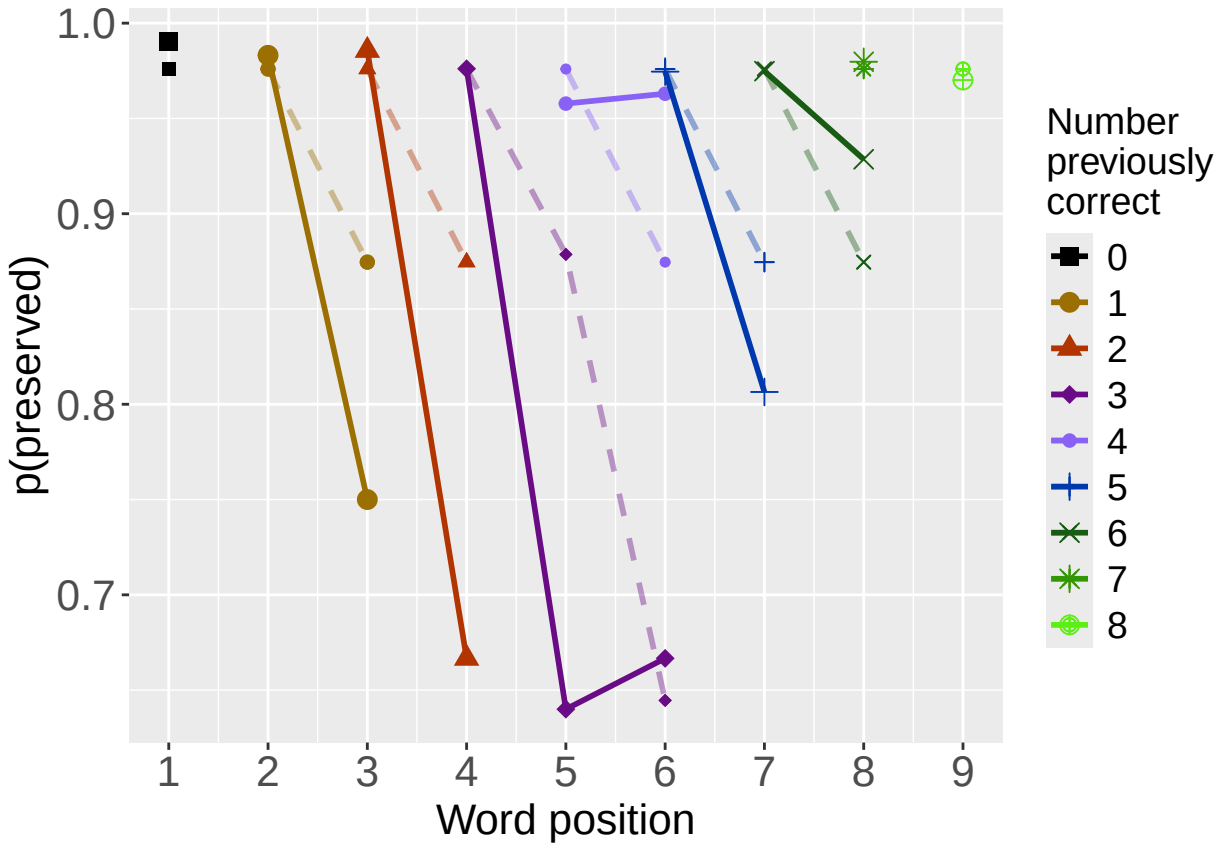
```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

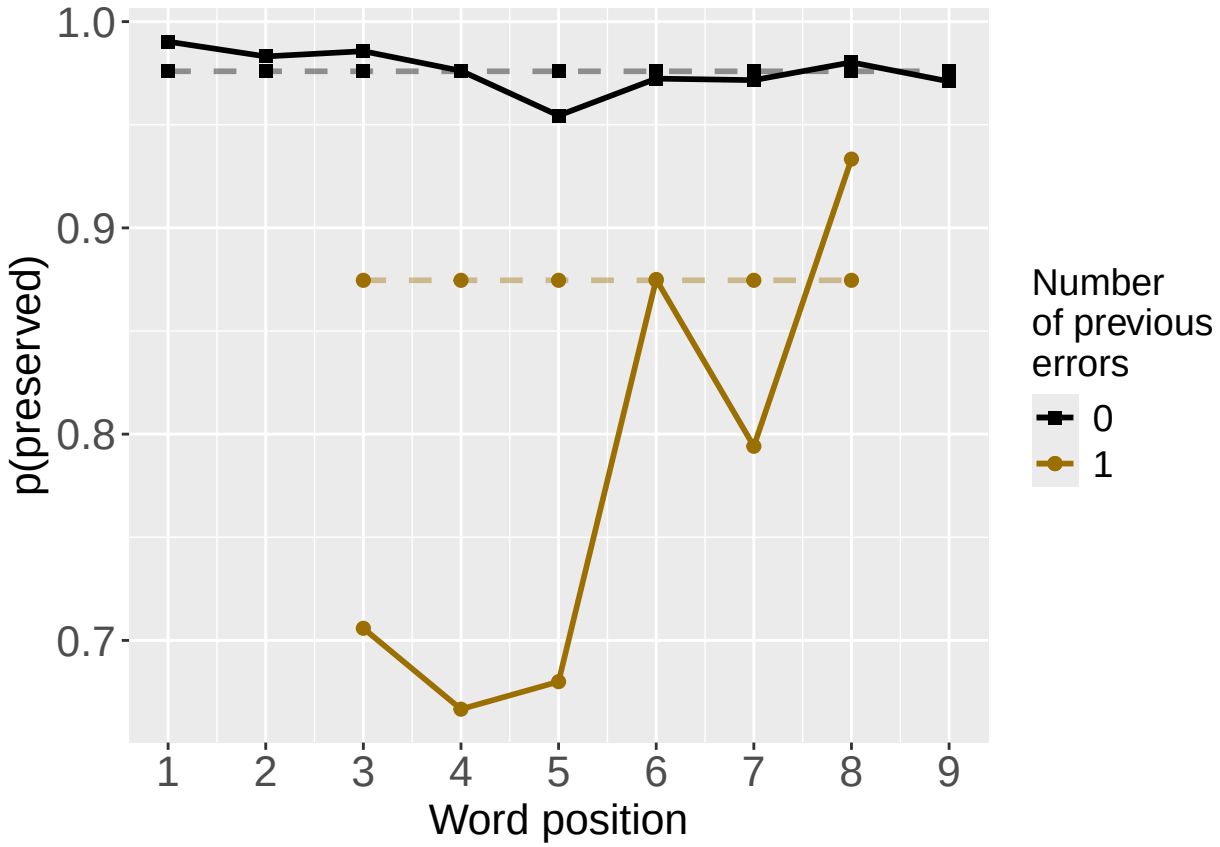
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                 CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                 CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```r
CumAICSummary <- NULL
```

```r
########
# level 2 -- Add position squared (quadratic with position)
########

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     I(pos^2)          pos
##     5.41166     -1.66924      0.07317     -0.78803
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       1348
## Residual Deviance: 1108  AIC: 1116
## log likelihood:  -554.1499
## Nagelkerke R2:  0.2010608
## % pres/err predicted correctly:  -253.6771
## % of predictable range [ (model-null)/(1-null) ]:  0.1537088
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.702       -1.760
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1128  AIC: 1132
## log likelihood:  -564.2311
## Nagelkerke R2:  0.1845804
## % pres/err predicted correctly:  -255.0358
## % of predictable range [ (model-null)/(1-null) ]:  0.1491937
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     5.39449      0.04961     -0.74107
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1292  AIC: 1298
## log likelihood:  -646.073
## Nagelkerke R2:  0.04795462
## % pres/err predicted correctly:  -296.3271
## % of predictable range [ (model-null)/(1-null) ]:  0.01198305
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	1116.300	0.00000	1.0000000	0.9996908	0.2010608	5.411659	-1.669243	0.0731741	-0.7880322
preserved ~ CumErr	1132.462	16.16237	0.0003093	0.0003092	0.1845804	3.702231	-1.760047	NA	NA
preserved ~ I(pos^2) + pos	1298.146	181.84614	0.0000000	0.0000000	0.0479546	5.394490	NA	0.0496100	-0.7410726

```
########
# level 2 -- Add length
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.702       -1.760
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1128  AIC: 1132
## log likelihood:  -564.2311
## Nagelkerke R2:  0.1845804
## % pres/err predicted correctly:  -255.0358
## % of predictable range [ (model-null)/(1-null) ]:  0.1491937
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      stimlen
##     3.90072     -1.74357     -0.02591
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1128  AIC: 1134
## log likelihood:  -564.1226
## Nagelkerke R2:  0.1847582
## % pres/err predicted correctly:  -255.1224
## % of predictable range [ (model-null)/(1-null) ]:  0.1489061
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      4.5900      -0.1636
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1338  AIC: 1342
## log likelihood:  -668.7569
## Nagelkerke R2:  0.009177715
## % pres/err predicted correctly:  -299.2348
## % of predictable range [ (model-null)/(1-null) ]:  0.002320514
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr	1132.462	0.000000	1.0000000	0.7091928	0.1845804	3.702231	-1.760047	NA
preserved ~ CumErr + stimlen	1134.245	1.782933	0.4100539	0.2908072	0.1847582	3.900720	-1.743568	-0.0259093
preserved ~ stimlen	1341.514	209.051549	0.0000000	0.0000000	0.0091777	4.590042	NA	-0.1636135

```
########
# level 2 -- add cumulative preserved
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres
##     3.93104     -1.73813     -0.07827
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1125  AIC: 1131
## log likelihood:  -562.5864
## Nagelkerke R2:  0.1872742
## % pres/err predicted correctly:  -255.402
## % of predictable range [ (model-null)/(1-null) ]:  0.147977
## ***************************
## model index:  1
##
```

```
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.702       -1.760
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1128  AIC: 1132
## log likelihood:  -564.2311
## Nagelkerke R2:  0.1845804
## % pres/err predicted correctly:  -255.0358
## % of predictable range [ (model-null)/(1-null) ]:  0.1491937
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      3.5165      -0.0739
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1344  AIC: 1348
## log likelihood:  -672.2345
## Nagelkerke R2:  0.00319751
## % pres/err predicted correctly:  -299.7032
## % of predictable range [ (model-null)/(1-null) ]:  0.0007639755
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	1131.173	0.000000	1.0000000	0.6558114	0.1872742	3.931035	-1.738131	-0.0782750
preserved ~ CumErr	1132.462	1.289367	0.5248286	0.3441886	0.1845804	3.702231	-1.760047	NA
preserved ~ CumPres	1348.469	217.296106	0.0000000	0.0000000	0.0031975	3.516533	NA	-0.0738972

```
########
# level 2 -- Add linear position (NOT quadratic)
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr          pos
##      4.1135      -1.6247      -0.1027
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1123  AIC: 1129
## log likelihood:  -561.3562
## Nagelkerke R2:  0.1892879
## % pres/err predicted correctly:  -255.3581
## % of predictable range [ (model-null)/(1-null) ]:  0.1481227
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.702       -1.760
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1128  AIC: 1132
## log likelihood:  -564.2311
## Nagelkerke R2:  0.1845804
## % pres/err predicted correctly:  -255.0358
## % of predictable range [ (model-null)/(1-null) ]:  0.1491937
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      4.4174      -0.2535
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1300  AIC: 1304
## log likelihood:  -650.1808
## Nagelkerke R2:  0.04096228
## % pres/err predicted correctly:  -296.6452
## % of predictable range [ (model-null)/(1-null) ]:  0.01092596
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr + pos	1128.712	0.000000	1.0000000	0.8670238	0.1892879	4.113477	-1.624691	-0.1027222
preserved ~ CumErr	1132.462	3.749792	0.1533709	0.1329762	0.1845804	3.702231	-1.760047	NA
preserved ~ pos	1304.362	175.649042	0.0000000	0.0000000	0.0409623	4.417355	NA	-0.2535385

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_main_effects_plus_one_model_summary.csv"),row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + I(pos^2) + pos	1116.300	0.000000	1.000000	0.9996908	0.2010665	3.8411659	-1.669243	0.0731741	0.7880322	NA	NA
preserved ~ CumErr + pos	1128.712	12.412000	1.000000	0.0867023	0.1892879	4.0113477	-1.624691	NA	0.1027222	NA	NA
preserved ~ CumErr + CumPres	1131.170	14.870000	1.000000	0.0255811	0.1872742	3.9931035	-1.738131	NA	NA	NA	-0.0782750
preserved ~ CumErr	1132.462	16.162372	2.000309	0.0003099	0.1845804	3.7702231	-1.760047	NA	NA	NA	NA
preserved ~ CumErr	1132.462	0.000000	1.000000	0.0709192	0.1845804	3.7702231	-1.760047	NA	NA	NA	NA
preserved ~ CumErr	1132.462	12.289367	0.524828	0.0344188	0.1845804	3.7702231	-1.760047	NA	NA	NA	NA
preserved ~ CumErr	1132.463	3.749792	0.153370	0.0132976	0.1845804	3.7702231	-1.760047	NA	NA	NA	NA
preserved ~ CumErr + stimlen	1134.245	5.782933	0.410053	0.0290807	0.1847582	3.9007207	-1.743568	NA	NA	-0.0259093	NA
preserved ~ I(pos^2) + pos	1298.146	81.846167	0.000000	0.0000000	0.0047954	6.6394490	NA	0.0496100	0.7410726	NA	NA
preserved ~ pos	1304.362	175.649042	0.000000	0.0000000	0.0040962	4.4173550	NA	NA	-0.2535385	NA	NA
preserved ~ stimlen	1341.512	209.051504	0.000000	0.0000000	0.0009177	4.7590042	NA	NA	NA	-0.1636135	NA
preserved ~ CumPres	1348.469	217.296106	0.000000	0.0000000	0.0003197	3.5516533	NA	NA	NA	NA	-0.0738972

```
# explore influence of frequency and length

if(grepl("stimlen",BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + log_freq")
  )
```

```r
}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
    Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + log_freq"),
      paste0(BestModelFormulaL2," + stimlen"),
      paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)
```

```
## ****************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     I(pos^2)          pos     log_freq
##     5.41427     -1.66814      0.07476     -0.79075      0.11252
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4393 Residual
## Null Deviance:       1348
## Residual Deviance: 1102  AIC: 1112
## log likelihood:  -551.1984
## Nagelkerke R2:  0.2058716
## % pres/err predicted correctly:  -253.4056
## % of predictable range [ (model-null)/(1-null) ]:  0.1546109
## ****************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     I(pos^2)          pos      stimlen     log_freq
##     5.17971     -1.67340      0.07335     -0.78764      0.03275      0.11746
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       1348
## Residual Deviance: 1102  AIC: 1114
## log likelihood:  -551.0736
## Nagelkerke R2:  0.2060748
## % pres/err predicted correctly:  -253.3296
## % of predictable range [ (model-null)/(1-null) ]:  0.1548634
## ****************************
## model index:  1
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     I(pos^2)          pos
##     5.41166     -1.66924      0.07317     -0.78803
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       1348
## Residual Deviance: 1108  AIC: 1116
## log likelihood:  -554.1499
## Nagelkerke R2:  0.2010608
## % pres/err predicted correctly:  -253.6771
## % of predictable range [ (model-null)/(1-null) ]:  0.1537088
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     I(pos^2)          pos      stimlen
##   5.4170992   -1.6691165    0.0732089   -0.7881001   -0.0007656
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4393 Residual
## Null Deviance:       1348
## Residual Deviance: 1108  AIC: 1118
## log likelihood:  -554.1499
## Nagelkerke R2:  0.2010609
## % pres/err predicted correctly:  -253.6783
## % of predictable range [ (model-null)/(1-null) ]:  0.1537047
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       3.303
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4397 Residual
## Null Deviance:       1348
## Residual Deviance: 1348  AIC: 1350
## log likelihood:  -674.0917
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -299.9332
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]
```

```
AICSummary<-data.frame(Model=Level3Res$Model,
                       AIC=Level3Res$AIC,
                       row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                          by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                            CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	log_freq	stimlen
preserved ~ CumErr + I(pos^2) + pos + log_freq	1112.397	0.000000	1.000000	0.6206968	0.2058751	6.414274	-1.668144	0.0747583	-0.7907483	0.1125153	NA
preserved ~ CumErr + I(pos^2) + pos + stimlen + log_freq	1114.147	1.750403	0.4167780	0.2586924	0.2060751	8.179712	-1.673405	0.0733472	-0.7876431	0.1174606	0.0327521
preserved ~ CumErr + I(pos^2) + pos	1116.308	3.910909	0.1420544	0.0881726	0.2010658	6.411659	-1.669243	0.0731741	-0.7880322	NA	NA
preserved ~ CumErr + I(pos^2) + pos + stimlen	1118.305	5.908946	0.0522627	0.0324392	0.2010659	9.417099	-1.669116	0.0732089	-0.7881001	NA	-0.0007656
preserved ~ 1	1350.182	237.785	1.000000e-52	6.206968e-53	0.0000000	0.302934	NA	NA	NA	NA	NA

```
# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions
```

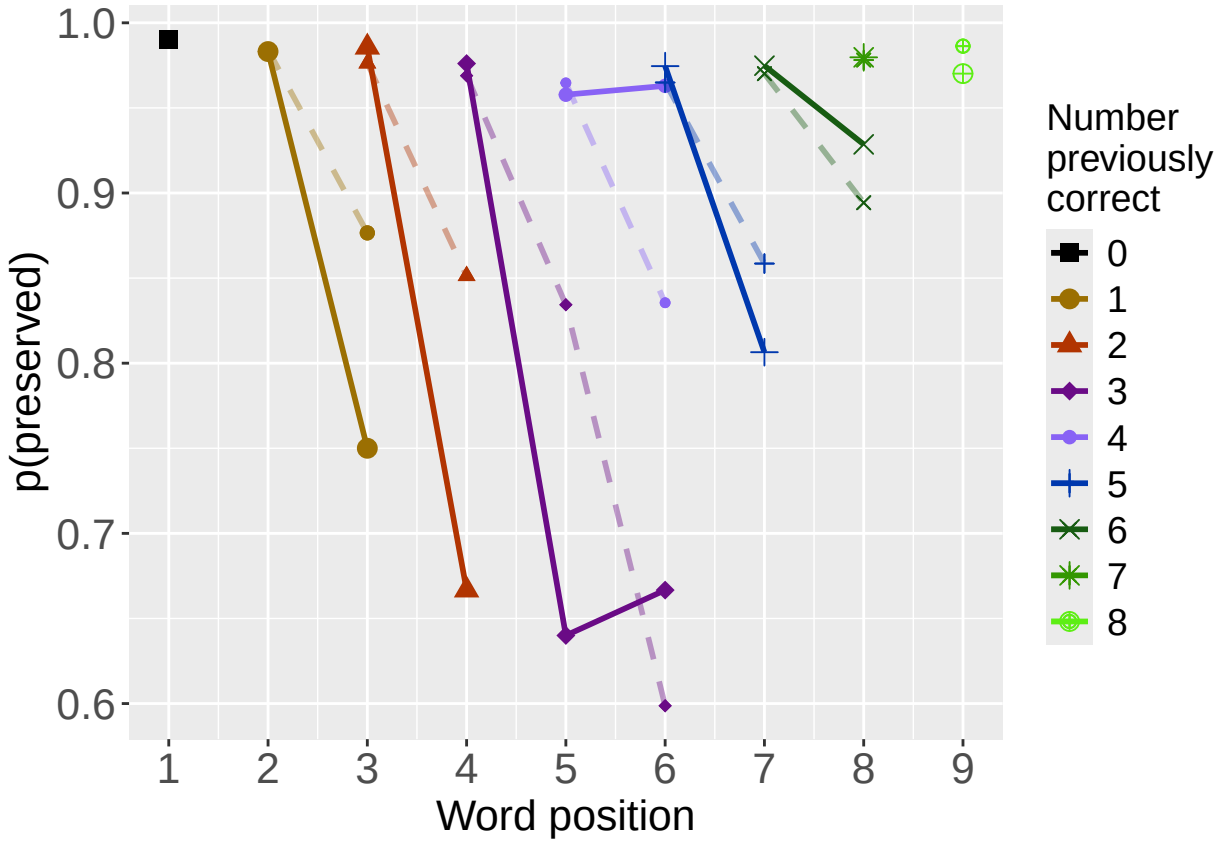
```
## Single term deletions
##
## Model:
## preserved ~ CumErr + I(pos^2) + pos + log_freq
##          Df Deviance    AIC
## CumErr    1   1286.4 1294.4
## pos       1   1120.8 1128.8
## I(pos^2)  1   1117.4 1125.4
## log_freq  1   1108.3 1116.3
## <none>        1102.4 1112.4
```

```
################################
# Single deletions from best model
################################

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                    RunLabel,"_",CurTask,
                                    "_best_model_single_term_deletions.csv"),
          row.names = TRUE)
```

```
# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

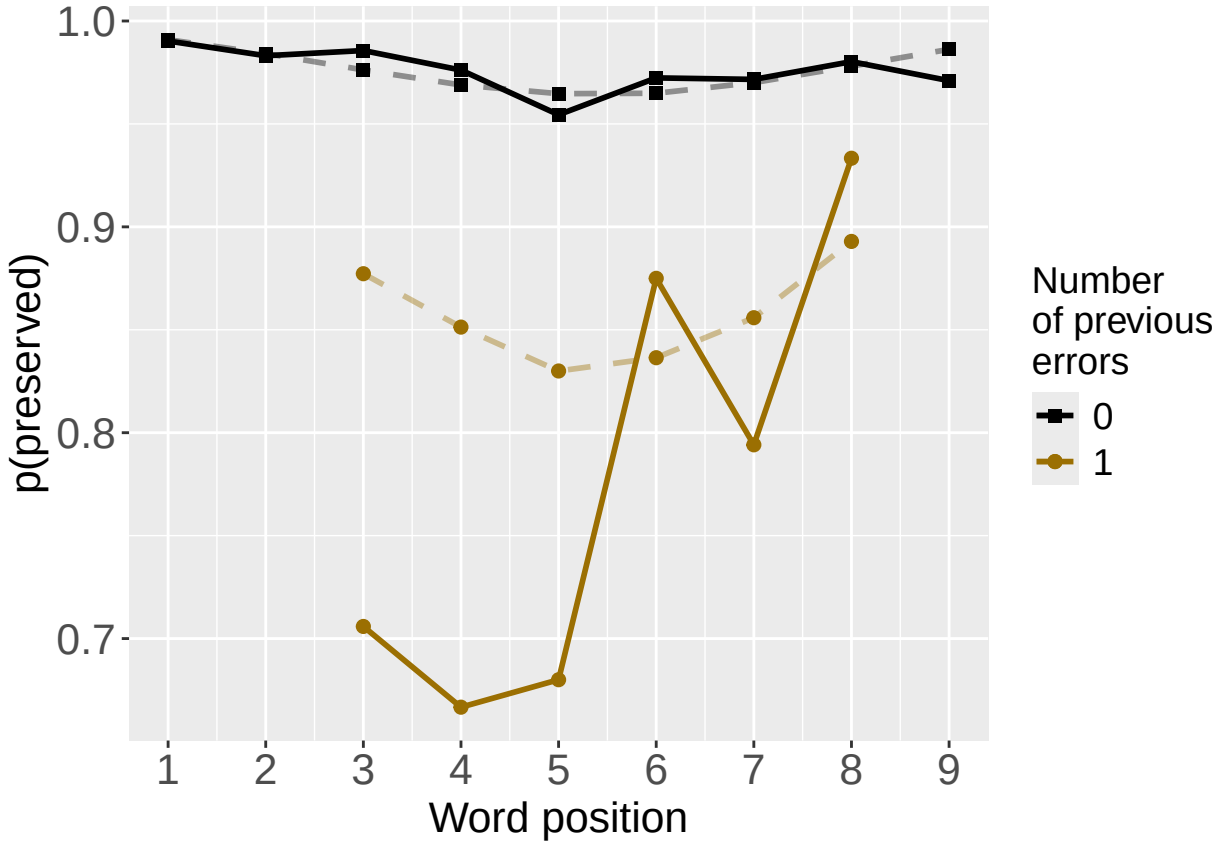
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",
                 RunLabel,"_",CurTask,
                 "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =
```

```
## Saving 6.5 x 4.5 in image
```

```
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "
```

```
## Saving 6.5 x 4.5 in image
```

```
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
  # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                        family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
```

```
  }
  ModelNames<-c(paste0("***",BestModelFormulaL3),
                rep(BestModelFormulaL3Rnd,RandomSamples))
  AICValues <- c(BestModelL3$aic,RndModelAIC)
  BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
  BestModelRndDF <- rbind(BestModelRndDF,
                          data.frame(Name=c("Random average"),
                                     AIC=c(mean(RndModelAIC))))
  BestModelRndDF <- rbind(BestModelRndDF,
                          data.frame(Name=c("Random SD"),
                                     AIC=c(sd(RndModelAIC))))
  write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                  "_best_model_with_random_cum_term.csv"),
            row.names = FALSE)
}
```

```
# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)
```

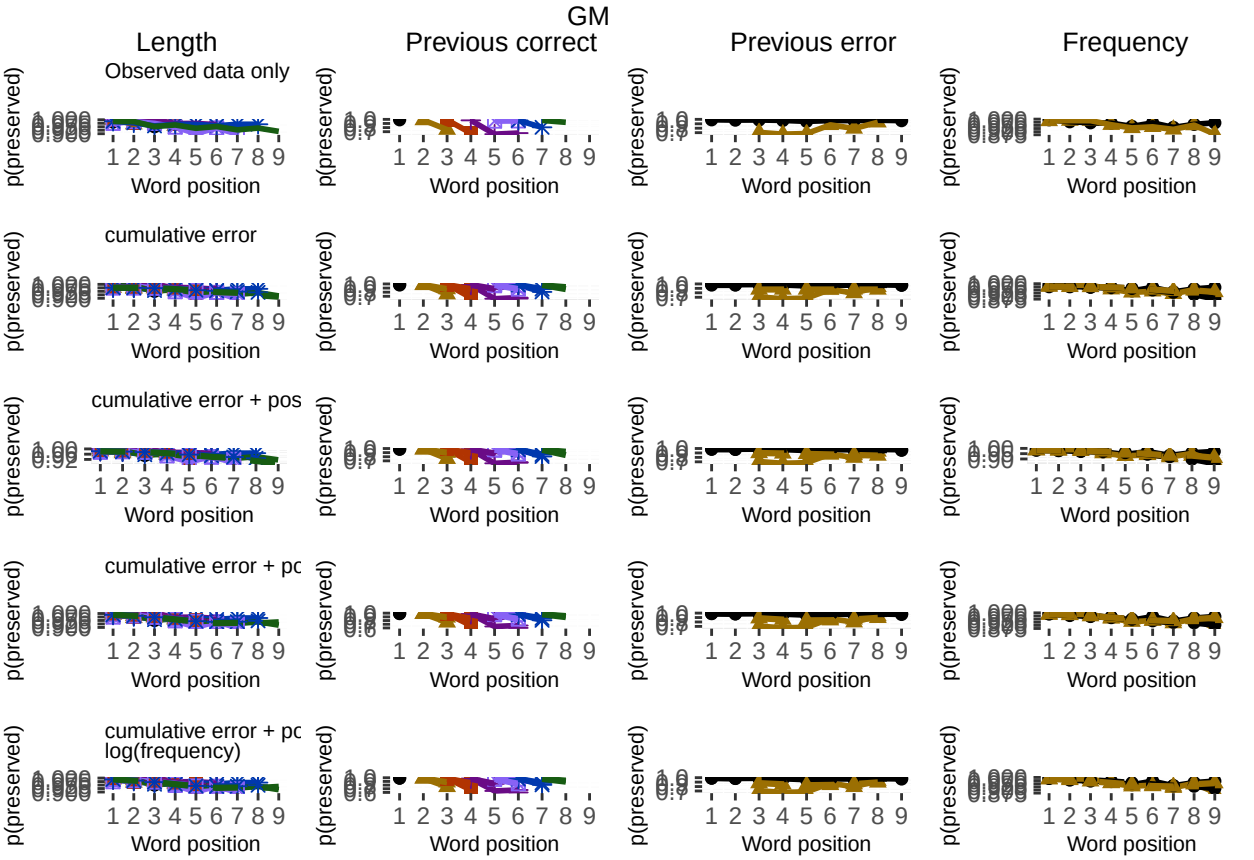
```
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.702       -1.760
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       1348
## Residual Deviance: 1128  AIC: 1132
## log likelihood:  -564.2311
## Nagelkerke R2:  0.1845804
## % pres/err predicted correctly:  -255.0358
## % of predictable range [ (model-null)/(1-null) ]:  0.1491937
```

```
## **************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr          pos
##      4.1135      -1.6247      -0.1027
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       1348
## Residual Deviance: 1123  AIC: 1129
## log likelihood:  -561.3562
## Nagelkerke R2:  0.1892879
## % pres/err predicted correctly:  -255.3581
## % of predictable range [ (model-null)/(1-null) ]:  0.1481227
## **************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr          pos     I(pos^2)
##     5.41166     -1.66924     -0.78803      0.07317
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       1348
## Residual Deviance: 1108  AIC: 1116
## log likelihood:  -554.1499
## Nagelkerke R2:  0.2010608
## % pres/err predicted correctly:  -253.6771
## % of predictable range [ (model-null)/(1-null) ]:  0.1537088
## **************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr          pos     I(pos^2)     log_freq
##     5.41427     -1.66814     -0.79075      0.07476      0.11252
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4393 Residual
## Null Deviance:       1348
## Residual Deviance: 1102  AIC: 1112
## log likelihood:  -551.1984
## Nagelkerke R2:  0.2058716
## % pres/err predicted correctly:  -253.4056
## % of predictable range [ (model-null)/(1-null) ]:  0.1546109
## **************************
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
```

```
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
ggsave(paste0(PlotName,".tif"),plot=FactorPlot,width = 360,height=400,units="mm",device="tiff",compressi
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```




```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                       CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	I(pos^2)	pos	log_freq
McFadden	0.1443712	0.0138234	0.0190681	0.0050467

	CumErr	I(pos^2)	pos	log_freq
SquaredCorrelation	0.0429515	0.0041589	0.0057345	0.0015082
Nagelkerke	0.1626861	0.0157526	0.0217202	0.0057126
Estrella	0.0476011	0.0044455	0.0061359	0.0016513

	deviance	deviance_explained
CumErr + pos + I(pos^2) + log_freq	1102.397	245.7865
CumErr + pos + I(pos^2)	1108.300	239.8834
CumErr + pos	1122.712	225.4708
CumErr	1128.462	219.7210
null	1348.183	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                                                 model deviance deviance_explained percent_explained
## CumErr + pos + I(pos^2) + log_freq CumErr + pos + I(pos^2) + log_freq 1102.397           245.7865          18.23094
## CumErr + pos + I(pos^2)                       CumErr + pos + I(pos^2) 1108.300           239.8834          17.79309
## CumErr + pos                                            CumErr + pos 1122.712           225.4708          16.72405
## CumErr                                                        CumErr 1128.462           219.7210          16.29756
## null                                                            null 1348.183             0.0000           0.00000
##                                    percent_of_explained_deviance increment_in_explained
## CumErr + pos + I(pos^2) + log_freq                     100.00000               2.401715
## CumErr + pos + I(pos^2)                                 97.59829               5.863861
## CumErr + pos                                            91.73442               2.339344
## CumErr                                                  89.39508              89.395080
## null                                                          NA               0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + pos + I(pos^2) + log_freq	18.23094	100.00000	2.401715
CumErr + pos + I(pos^2)	17.79309	97.59829	5.863861
CumErr + pos	16.72405	91.73442	2.339344
CumErr	16.29756	89.39508	89.395080
null	0.00000	NA	0.000000

```
                                  "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents
```

```
##           Nagelkerke
## CumErr    0.79023118
## I(pos^2) 0.07651650
## pos       0.10550388
## log_freq 0.02774844
```

```
sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)
```



```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```



```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_sse_results_table.csv"),row.names = TRUE)
sse_table
```

```
##                                    model p_accounted_for model_deviance diff_CumErr+pos diff_CumErr diff_CumErr+pos+I(pos^2)
## 1                  preserved ~ CumErr+pos       0.8594814       1122.712      0.00000000 -0.01689514             -0.040109624
## 2                      preserved ~ CumErr       0.8763766       1128.462      0.01689514  0.00000000             -0.023214485
## 3          preserved ~ CumErr+pos+I(pos^2)       0.8995911       1108.300      0.04010962  0.02321448              0.000000000
```

model	p_accounted_for	model_deviance
preserved ~ CumErr+pos	0.8594814	1122.712
preserved ~ CumErr	0.8763766	1128.462
preserved ~ CumErr+pos+I(pos^2)	0.8995911	1108.300
preserved ~ CumErr+pos+I(pos^2)+log_freq	0.9019945	1102.397

model	diff_CumErr+pos	diff_CumErr	diff_CumErr+pos+I(pos^2)	diff_CumErr+pos+I(pos^2)+log_freq
preserved ~ CumErr+pos	0.0000000	-0.0168951	-0.0401096	-0.0425131
preserved ~ CumErr	0.0168951	0.0000000	-0.0232145	-0.0256179
preserved ~ CumErr+pos+I(pos^2)	0.0401096	0.0232145	0.0000000	-0.0024035
preserved ~ CumErr+pos+I(pos^2)+log_freq	0.0425131	0.0256179	0.0024035	0.0000000

```
## 4 preserved ~ CumErr+pos+I(pos^2)+log_freq       0.9019945       1102.397       0.04251308  0.02561794                0.002403452
##   diff_CumErr+pos+I(pos^2)+log_freq
## 1                      -0.042513076
## 2                      -0.025617936
## 3                      -0.002403452
## 4                       0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                   CurTask,"_results_report_df.csv"),row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```